Public Key Crypto notes - 08

By contributors

March 30. 2026

1 Oblivious transfer recap

A sends m_0, m_1, B chooses $b \in_R \{0,1\}$ and obtains m_b. RSA-based protocol.

2 Protocol 2 – Bellari-Micali

- Diffie-Hellman-based, interactive protocol.
- Let p, g, q be as in ElGamal.
- A chooses $c \in_R < g >$, where $< g >= \{g^i : i = 0, \ldots, q-1\}$ and sends c to B.
- B chooses $k \in_R \mathbb{Z}_q, b \in_R \{0,1\}, y_b = g^k, y_{1-b} = \frac{c}{g^k}$, and sends y_0, y_1 to A.
- A checks $y_0 \cdot y_1 \stackrel{?}{=} c$ and chooses two random exponents $r_0, r_1 \in_R \mathbb{Z}_q$
  - $c_0 = (g^{r_0}, H(y_0^{r_0}) \oplus m_0)$
  - $c_1 = (g^{r_1}, H(y_1^{r_1}) \oplus m_1)$
  - A sends c_0, c_1 to B
- B calculates $c_b = (v_0, v_1), m_b = H(v_0^k) \oplus v_1$

3 Application of oblivious transfers

3.1 Secure multiparty computations (MPC)

- There are two parties A, B. Both of them have a secret input x_A, x_B. There is a function $f(x, x')$. A and B want to evaluate $f(x_A, x_B)$ without them learning the other's secret.
- Example: Dating protocol
  - Two parties: A, B
  - Let $d_A = \begin{cases} 1 \text{ if } A \text{ wants to date with B} \\ 0 \text{ otherwise} \end{cases}$ and, $d_B = \begin{cases} 1 \text{ if } B \text{ wants to date with A} \\ 0 \text{ otherwise} \end{cases}$
  - The function to evaluate this: $\text{AND}(d_A, d_B)$
- Evaluating AND with oblivious transfer:
  - If $d_A = 1$, let $m_0 = 0$ (meaning that B chose 0), $m_1 = 1$
  - If $d_A = 0$, let $m_0 = 0, m_1 = 0$
  - Then, A sends m_0, m_1 to B by oblivious transfer

3.2 Group signatures

- Group/organization
- A Group Manager can add users to a group
- The group $(S_0, S_1, \dots S_n)$ jointly issues signature (m, σ) on behalf of the group itself
- The exact individual who actually made the signature is masked
- Requirements:
  - σ cannot be linked to any S_i
  - The Group Manager M can resolve σ (i.e. can learn the actual signer S_i)
- Algorithms:
  - Setup: M sets the parameters up
  - Joining member S_i is added by M (can even happen dynamically)
  - Signing: S_i signs m and outputs σ
  - Verification: Third-party user U verifies by $\mathsf{Ver}(m, \sigma)$
  - Open: M links the signature σ to S_i

3.2.1 Example of group signature protocols

Chaum's group signature, '91:

- Setup/Join:
  - S_i generates ElGamal keys: $pk_i = (p, g, q, g^{x_i}), sk_i = x_i$
  - S_i gives pk_i to M
- Sign:
  - At period t, M chooses $r_{t,i} \in_R \mathbb{Z}_q$, and sends it to S_i
  - M publishes $P_t = \{g^{r_{t,i} \cdot x_i}, i = 1, 2, \dots\}$ (all blinded public keys at period t)
  - S_i uses $r_{t,i} \cdot x_i$ as a secret key for the ElGamal signature, $pk = g^{r_{t,i} \cdot x_i}$.
- Verify:
  - U checks: $pk \stackrel{?}{\in} P_t$ and verifies the ElGamal signature.
- Open:
  - M checks for which $i : pk = (g^{x_i})^{r_{t,i}}$

Remark 3.1. At period t, only *one* signer can make a signature. Hence, the whole signature (m, σ, P_t) is linear in the size of the group.

Before protocol 2, we need preparation with proof of knowledge protocols.

3.3 Proof of knowledge protocols

3.3.1 Schnorr's PoK protocol

- Schnorr PoK$[x : g^x = h](m)$
- Prover chooses $k \in_R \mathbb{Z}_q$, sends $I = g^k$ to the Verifier.
- Verifier chooses $r \in_R \mathbb{Z}_q$ or $r = H(I)$ or $r = H(I||m)$, and sends r to the Prover
- Prover calculates $s = rx + k$, and sends s to the Verifier
- The Verifier checks whether $r \stackrel{?}{=} H(g^s \cdot h^{-r}||m)$, where $g^s \cdot h^{-r} = I$

Remark 3.2. Success probability without knowing x is $\approx \frac{1}{q}$.

3.3.2 PoK of LogLog

Goal: Prover knows x such that $y = g^{(a^x)}$ with given g, y, a.

- Prover chooses $k \in_R \{0, \dots, 2^\lambda\}, t = g^{a^k}$ and t is sent to the Verifier
- The Verifier chooses $r \in_R \{0, 1\}$ and sends it to the Prover
- The Prover calculates: $s = \begin{cases} k, \text{ if } r = 0 \\ k - x, \text{ if } r = 1 \end{cases}$ and sends s to the Verifier
- Verifier performs the following: $t = \begin{cases} g^{(a^s)}, \text{ if } r = 0 \\ y^{(a^s)}, \text{ if } r = 1 \end{cases}$
- Correctness:
  - $r = 0$ ✓
  - $r = 1 \rightarrow y^{(a^s)} = y^{(a^{k-x})} = [g^{(a^x)}]^{(a^{k-x})} = g^{(a^x)\cdot(a^k)\cdot(a^{-x})} = g^{(a^k)} = t$

3.3.3 Remarks

Remark 3.3. If $r = 0$, the Verifier checks if t has the right form. If $r = 1$, then the Verifier checks if P knows x, provided t has the right form.

Remark 3.4. Success probability is $\frac{1}{2}$.

3.3.4 Non-interactive PoK protocol

PoK$[x : y = g^{(a^x)}](m = r, s_1, \dots, s_l) \in \{0, B^l \cdot \mathbb{Z}_q^l\}$ such that $r = H(m||y||g||a||t_1|| \dots ||t_l)$ with $t_i = \begin{cases} g^{(a^{s_i})}, \text{ if } r_i = 0 \\ y^{(a^{s_i})}, \text{ if } r_i = 1. \end{cases}$
Construction:

- $t_i = g^{(a^{k_i})}$ with random k_i and $s_i = \begin{cases} k_i, \text{ if } r_i = 0 \\ k_{i-x}, \text{ if } r_i = 1 \end{cases}$.